Numbers In Pi

AlgoExpert · Hard · Dynamic Programming

The problem

- Minimum spaces to split the digit string into numbers all from the given list, or -1.
- DP over suffixes.

Intuition

- Find the fewest pieces to cover each suffix of the digits with favourite numbers, using memoisation.

Brute force vs optimal

- Try every split → exponential
- Memoised suffix DP → $O(n^2 + m)$
- Hash set for O(1) lookups

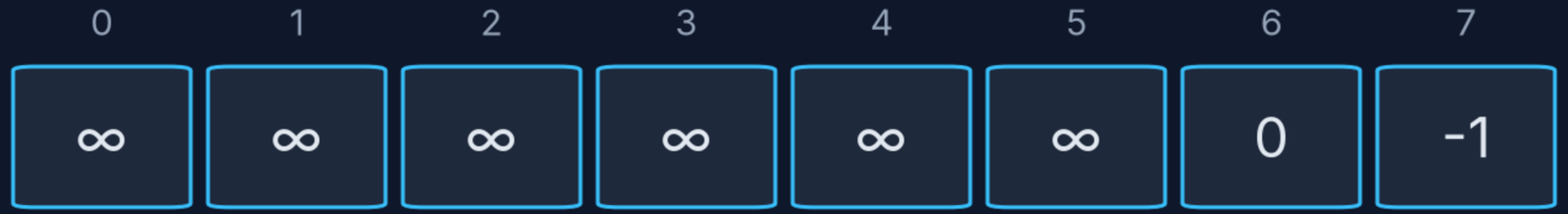
Approach

- 1. Put numbers in a set
- 2. From each start, try every prefix in the set
- 3. Recurse on the rest with memo
- 4. Spaces equal pieces minus one

Worked example



min spaces from i

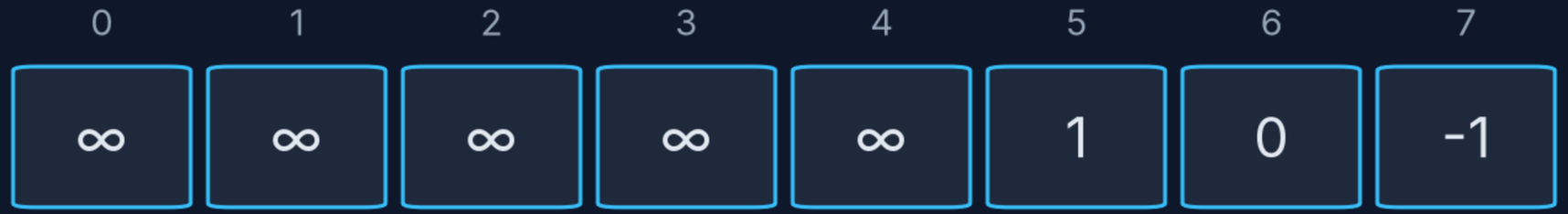


i=6: 0

Worked example



min spaces from i

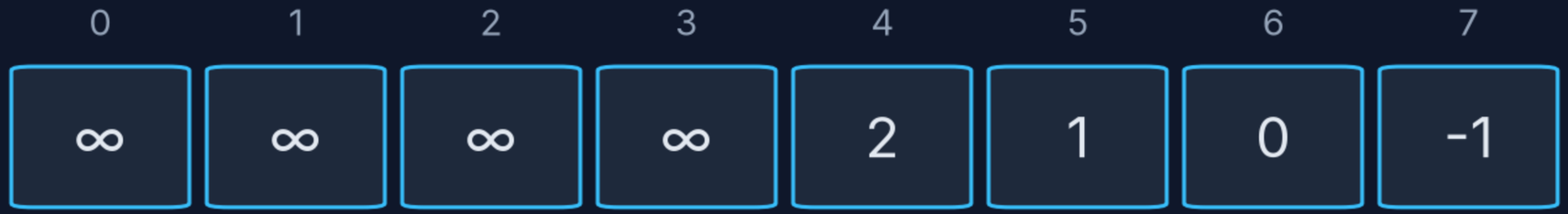


i=5: 1

Worked example



min spaces from i

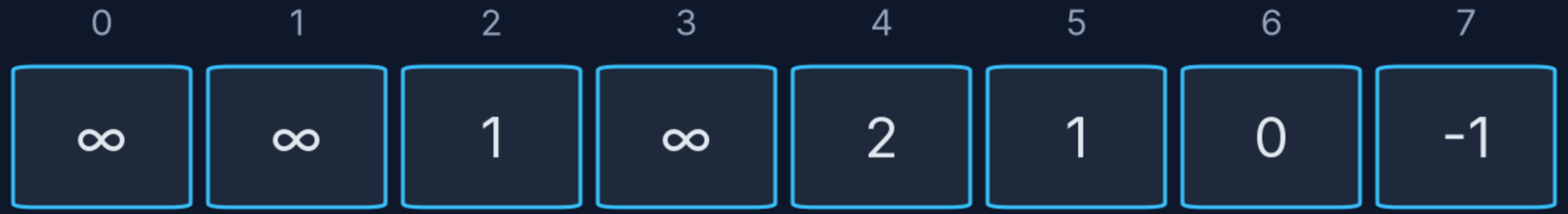


i=4: 2

Worked example



min spaces from i

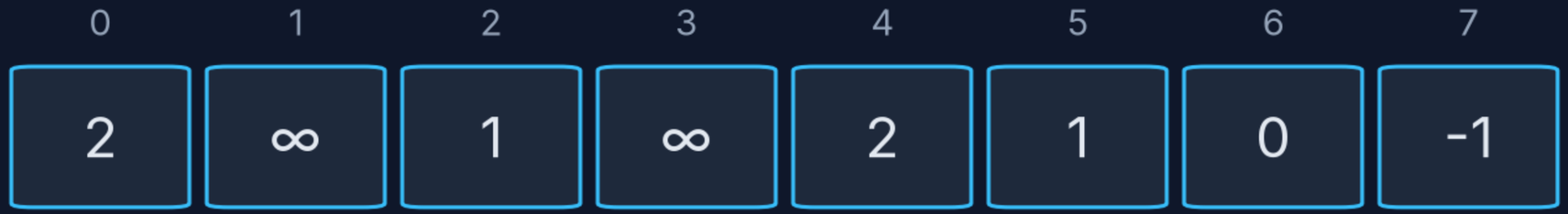


i=2: 1

Worked example



min spaces from i



i=0: 2

Java solution

```java
public class NumbersInPi {
    public static int numbersInPi(String pi, String[] numbers) {
        Set<String> set = new HashSet<>(Arrays.asList(numbers));
        int n = pi.length();
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[n] = -1;
        for (int i = n - 1; i >= 0; i--)
            for (int j = i + 1; j <= n; j++)
                if (dp[j] != Integer.MAX_VALUE && set.contains(pi.substring(i, j))) dp[i] = Math.min(dp[i], dp[j] + 1);
        return dp[0] == Integer.MAX_VALUE ? -1 : dp[0];
    }
}
```

Complexity

- Time: O(n^3 + m)
- Space: O(n + m)

Edge cases & pitfalls

- Dp[n] = −1 (no space after the last number)
- Impossible → −1
- Substring cost

Interview tips & follow-ups

- Same as Word Break (min pieces)

Recap

- Find the fewest pieces to cover each suffix of the digits with favourite numbers, using memoisation.
- Complexity: O(n^3 + m)